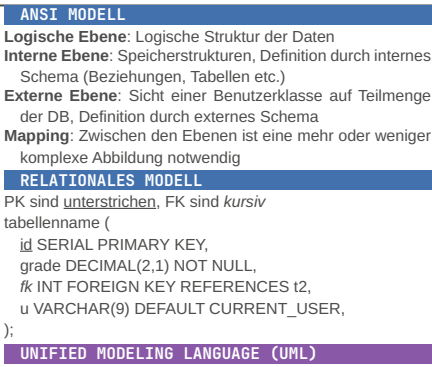




UNIFIED MODELING LANGUAGE (UML)			
Eins		Mehrere	
Eins (zwingend)		Mehrere (mind. 1)	
0 oder eins		0 oder mehrere	
Assoziation		Komposition	
Aggregation		Vererbung	

NORMALISIERUNG

1NF: Atomare Attributwerte: *track* aufteilen

2NF: Nichtschlüsselattr. voll vom Schlüssel abhängig.
Ist PK atomar, dann 2NF gegeben. Im Beispiel sind nicht alle Attribute des PK notwendig, um *album* eindeutig zu identifizieren

⇒ track	cd													
<table><tr><th>track</th><th>cd_id</th><th>titel</th></tr><tr><td>1</td><td>1</td><td>Turnover</td></tr><tr><td>2</td><td>1</td><td>Song #1</td></tr></table>	track	cd_id	titel	1	1	Turnover	2	1	Song #1	<table><tr><th>id</th><th>album</th></tr><tr><td>1</td><td>Repeater</td></tr></table>	id	album	1	Repeater
track	cd_id	titel												
1	1	Turnover												
2	1	Song #1												
id	album													
1	Repeater													

<i>id</i>	<i>album</i>	<i>interpret</i>	<i>land</i>
1	Repeater	Fugazi	USA
2	Red Medicine	Fugazi	USA

BCNF: Nur abhängigkeiten vom Schlüssel
(Voll-)funktionale Abhängigkeit: B hängt von A ab, zu jedem Wert von A gibt es genau einen Wert von B ($A \rightarrow B$)
Teilweise funkt. Abh.: B hängt von A ab, aber auch von einem Teil eines zusammengesetzten Schlüssels.
Transitive Abhängigkeit: B hängt vom Attribut A ab, C hängt von B ab ($A \rightarrow B \wedge B \rightarrow C \Rightarrow A \rightarrow C$)
Denormalisierung: In geringere NF zurückführen (Verbessert Performance und reduziert Joins-Komplexität)

Einfügeanomalie, Löschanomalie, Änderungsanomalie

```

Falls mit GRANT OPTION: Der Berechtigte kann den Zugriff
anderen Usern verteilen. → REVOKE ... CASCADE;

CREATE ROLE u WITH LOGIN PASSWORD ''; -- user
GRANT INSERT ON TABLE t TO u WITH GRANT OPTION;
ALTER ROLE u CREATOROLE, CREATEDB, INHERIT;
CREATE ROLE r; -- group
GRANT r TO u; -- put user u in group r
REVOKE CREATE ON SCHEMA s FROM r;
CREATE ROLE u PASSWORD ' ' IN ROLE r; -- equivalent

```

```

ROW-LEVEL SECURITY (RLS)

CREATE TABLE exams (
  id SERIAL, -- other fields...
  teacher VARCHAR(60) DEFAULT current_user
);

CREATE POLICY teachers_see_own_exams ON exams
FOR ALL TO PUBLIC USING (teacher = current_user);

ALTER TABLE exams ENABLE ROW LEVEL SECURITY;

DATA DEFINITION LANGUAGE (DDL)

```

Wichtig: NOT NULL wo notwendig nicht vergessen

```
Tabelle pro Sub- und Superklasse:
```

```
CREATE TABLE sup ( -- 3.a
    id SERIAL PRIMARY KEY,
    name TEXT UNIQUE
);
CREATE TABLE sub1 (
    id SERIAL PRIMARY KEY,
    age INT
);
CREATE TABLE sub2 (
    id SERIAL PRIMARY KEY
);
ALTER TABLE sub1 ADD CONSTRAINT id FOREIGN KEY
REFERENCES sup (id);
ALTER TABLE sub2 ADD CONSTRAINT id FOREIGN KEY
REFERENCES sup (id);
```

Einzige Tabelle für Superklasse: Enthält alle Attribute

```
CREATE TABLE a_b(  
  a INTEGER REFERENCES a(id),  
  b INTEGER REFERENCES b(id),  
  PRIMARY KEY(a, b)  
);
```

```
CREATE VIEW v (id, u) AS SELECT id, u FROM t;
-- complex query

CREATE VIEW cheap_restaurant_view AS
WITH big_restaurant AS (
    SELECT * FROM restaurant
    WHERE anzahl_plaetze >= 20
)
SELECT r.name AS restaurant_name, s.name,
       MIN(g.preis) AS cheap_gericht
FROM big_restaurant r
LEFT JOIN skigebiet s ON (s.id = r.skigebiet_id)
LEFT JOIN menukarte m ON (r.id = m.restaurant_id)
LEFT JOIN menu_gericht mg ON (m.id = mg.menu_id)
LEFT JOIN gericht g ON (g.id = mg.gericht_id)
WHERE ist_tagesmenu = true
GROUP BY r.id, s.id, restaurant_name
HAVING MIN(g.preis) >= 3
ORDER BY cheap_gericht;
```

MATERIALIZED VIEW
Speichert resultat auf Disk

TEMPORÄRE TABELLEN

```
CREATE TEMPORARY TABLE temp_products (  
  id SERIAL PRIMARY KEY,  
  product_name TEXT  
);
```

```
SELECT ts.product_name, ts.quantity FROM
temp_sales ts JOIN temp_products tp ON
ts.product_name = tp.product_name;
```

```
DREIWERTEIGE LOGIK (CURSED)
SELECT NULL IS NULL; -- true
SELECT NULL = NULL; -- [unknown]
```

CASTING

Explicit

```
CAST(5 AS float8) = 5::float8
SELECT 'ABCDEFG'::NUMERIC; -- error
SELECT SAFE_CAST('ABCDEFG' AS NUMERIC); -- NULL
```

```
DATA MANIPULATION LANGUAGE (DML)

SELECT ::= [ 'WITH' | 'RECURSIVE' ] { <with_query> [ ',' ... ] }
SELECT { 'ALL' | 'DISTINCT' | 'ON' | <expression>
[ ',' ... ] }
[ { '*' | <expression> [ 'AS' ] <output_name> }
[ ',' ... ] ]
[ 'FROM' <from_item> [ ',' ... ] ]
[ 'WHERE' <condition> ]
[ 'GROUP BY' { 'ALL' | 'DISTINCT' } <grouping_elem>
[ ',' ... ] ]
[ 'HAVING' <condition> ]
[ 'WINDOW' <window_name> 'AS' { <window_def> } [ ',' ... ] ]
[ { 'UNION' | 'INTERSECT' | 'EXCEPT' } [ 'ALL' | 'DISTINCT' ]
[ ',' ... ] ]
SELECT {
[ 'ORDER BY' <expression> [ 'ASC' | 'DESC' ] 'USING' <op> ]
[ 'NULLS' { 'FIRST' | 'LAST' } [ ',' ... ] ]
[ 'LIMIT' { <count> | 'ROW' ]
[ 'OFFSET' <start> [ 'ROW' ] 'ROWS' ] ]
```

```

from_item ::= <table> [ '*' ] [ [ 'AS' ] <alias> [ ( [ <col_alias>
['', ... ] ] ) ]
[ [ 'LATERAL' ] ( [ <select> [ [ 'AS' ] <alias> [ ( [ <col_alias>
['', ... ] ] ) ] ] ] ]
<with_query_name> [ [ 'AS' ] <alias> [ ( [ <col_alias>
['', ... ] ] ) ] ]
<from_item> <join_type> <from_item> [ [ 'ON' ] <join_condition> ]
[ [ 'USING' ] ( [ <join_column> [ ... ] ] ) ] 'AS' <join_using_alias> ] ]
<from_item> 'NATURAL' <join_type> <from_item>
<from_item> 'CROSS JOIN' <from_item>

[with_query ::= <name> [ ( [ <col_name> [ ' ' ... ] ] ) ]
[ 'AS' ] <select> [ <values> [ <insert> [ <update> [ <delete> ]
[merge] ] ] ]
[ [ 'USING' ] <cycle_path_col_name> ]

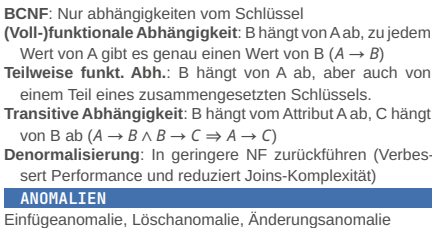
```

JOINS

users (*u*) actions (*a*) **INFO:** FK *uid* in den Query-Resultaten unten aus Platzgründen ausgelassen

<i>id</i>	<i>name</i>
1	Alice
2	Bob

<i>id</i>	<i>uid</i>	<i>action</i>
7	1	LOGIN
8	2	VIEW
9	4	LOGIN



DATABASE SYSTEM (DBS)	
Besteht aus DBMS und Datenbasen	
DATABASE	MANAGEMENT SYSTEM (DBMS)
– (A) Transaktionen – (C) Konsistenz – (I) Mehrbenutzerbetrieb – (D) Grosse Datenmengen – (D) Sicherheit	– Datentypen – Abfragesprache – Backup & Recovery – Redundanzfreiheit – Kapelung

